

IKS IN COMPUTATIONAL SYSTEMS

1. Title: "IKS Concept as CS Concept"

Pingala's Chandaḥśāstra as Binary Encoding and Combinatorial Generation

IKS Concept: Pingala's Chandaḥśāstra

CS Concept: Binary Encoding and Combinatorial Generation

Implementation Language: Python

2. Problem Statement

IKS Concept as CS Concept

Pingala's *Chandaḥśāstra* is an important work associated with the study of Sanskrit prosody (*Chandas*). It describes the arrangement and combinations of syllables in poetic metres. Two fundamental syllable classifications used in Sanskrit prosody are **Laghu** (light/short) and **Guru** (heavy/long).

In this project, the two possible syllable states are represented computationally as binary values:

Laghu (L) $\rightarrow 0$
Guru (G) $\rightarrow 1$

For a sequence containing n syllable positions, each position can have two possible states: Laghu or Guru.

Therefore, the total number of possible patterns is:

[
 2^n
]

The objective of this implementation is to develop a Python-based system that:

1. Accepts the number of syllable positions as input.
2. Generates all possible Laghu-Guru combinations.
3. Represents each combination using binary encoding.
4. Converts the binary representation into decimal form.
5. Allows a user to convert and search a particular Laghu-Guru pattern.
6. Validates invalid inputs.
7. Demonstrates the relationship between an Indian Knowledge System concept and modern computational concepts.

Problem Statement

To implement Pingala's Chandaḥśāstra's Laghu-Guru combinatorial representation as a modern binary encoding and combinatorial generation system using Python.

Important Conceptual Note

The project uses **binary encoding as a modern computational model** for the two-state Laghu-Guru structure. It does not claim that Pingala used modern binary notation in the same form as a present-day computer system.

3. Conceptual Mapping Table

Pingala's Chandaḥśāstra Concept	Computer Science Concept
Chandaḥśāstra	Computational representation
Laghu	Binary 0
Guru	Binary 1
Syllable	Bit position
Laghu-Guru sequence	Binary string
Possible Laghu-Guru patterns	Combinatorial generation
Number of possible patterns	(2^n)
Pattern conversion	Binary encoding
Pattern identification	Searching
Rules for generating patterns	Algorithm
Number of syllable positions	Input size
Generated combinations	Computational output

Example Mapping

For three syllable positions:

Laghu-Guru:

LLL
LLG
LGL
LGG
GLL
GLG
GGL
GGG

Using:

L = 0
G = 1

the corresponding binary representations are:

000
001
010
011
100
101
110
111

Thus:

$[2^3 = 8]$

possible patterns are generated.

4. Algorithm Specification (Pseudo-code)

4.1 Rules

The implementation follows these rules:

Rule 1 — Laghu Representation

$L \rightarrow 0$

Rule 2 — Guru Representation

$G \rightarrow 1$

Rule 3 — Pattern Length

For n syllable positions, every generated pattern must contain exactly n symbols.

Rule 4 — Number of Patterns

For n positions:

[
Total\ Patterns = 2^n
]

Rule 5 — Binary Generation

All binary numbers from:

[
0
]

to:

[
 $2^n - 1$
]

are generated.

Rule 6 — Binary-to-Laghu/Guru Conversion

$0 \rightarrow L$
 $1 \rightarrow G$

Rule 7 — Pattern Validation

Only L and G are accepted as valid pattern symbols.

4.2 Pattern Generation Algorithm

```
START

INPUT n

IF n is not a positive integer:
    DISPLAY "Invalid input"
    STOP

total_patterns ← 2^n

FOR number ← 0 TO total_patterns - 1:
    binary ← Convert number to n-bit binary
    pattern ← empty string
    FOR each bit in binary:
        IF bit = 0:
            append "L" to pattern
        ELSE:
            append "G" to pattern
    DISPLAY pattern
    DISPLAY binary
    DISPLAY decimal number
END FOR

STOP
```

4.3 Pattern Conversion Algorithm

```
START

INPUT Laghu-Guru pattern

Remove spaces
Convert pattern to uppercase

IF pattern is empty:
    DISPLAY "Invalid pattern"
    STOP

FOR each symbol:
    IF symbol = "L":
        convert to "0"
    ELSE IF symbol = "G":
        convert to "1"
    ELSE:
        DISPLAY "Invalid pattern"
        STOP

Convert binary string to decimal

DISPLAY pattern
```

```
DISPLAY binary representation
DISPLAY decimal representation

STOP
```

4.4 Example

Input:
n = 3

The system calculates:

[
2^3 = 8
]

patterns.

The generated values are:

Decimal	Binary	Laghu-Guru
0	000	LLL
1	001	LLG
2	010	LGL
3	011	LGG
4	100	GLL
5	101	GLG
6	110	GGL
7	111	GGG

5. Working Code

The implementation is developed in Python.

The source code is maintained separately from this report.

Project Structure

```
Pingala-Chandas-IKS/
├── src/
│   ├── main.py
│   └── chandas.py
├── tests/
│   └── test_chandas.py
├── README.md
└── requirements.txt
```

Main Functions

The implementation provides the following functionality:

`generate_patterns(n)`

Generates all possible Laghu-Guru patterns.

`pattern_to_binary(pattern)`

Converts a Laghu-Guru pattern into binary representation.

`pattern_to_decimal(pattern)`

Converts a Laghu-Guru pattern into its decimal value.

`total_patterns(n)`

Calculates the total number of possible patterns using:

```
[
2^n
]
```

Sample Working

For input:

Enter number of syllables: 3

the program produces:

Pattern	Binary	Decimal
LLL	000	0

LLG	001	1
LGL	010	2
LGG	011	3
GLL	100	4
GLG	101	5
GGL	110	6
GGG	111	7

Pattern Search Example

Input:

Enter Laghu-Guru pattern: LGLG

Output:

Pattern : LGLG
Binary : 0101
Decimal : 5

6. Minimum 10 Test Cases

The implementation was tested using valid, boundary, and invalid inputs.

Test Case	Input	Expected Result	Status
1	n = 1	2 patterns generated	Pass
2	n = 2	4 patterns generated	Pass
3	n = 3	8 patterns generated	Pass
4	n = 4	16 patterns generated	Pass
5	n = 5	32 patterns generated	Pass
6	n = 6	64 patterns generated	Pass
7	L	Binary 0, Decimal 0	Pass
8	G	Binary 1, Decimal 1	Pass
9	LG	Binary 01, Decimal 1	Pass
10	GLG	Binary 101, Decimal 5	Pass
11	LGLG	Binary 0101, Decimal 5	Pass
12	GGGG	Binary 1111, Decimal 15	Pass
13	LXL	Invalid pattern	Pass
14	Empty pattern	Invalid pattern	Pass
15	n = 0	Invalid input	Pass
16	n = -3	Invalid input	Pass
17	n = 7	128 patterns generated	Pass
18	GGGL	Binary 1110, Decimal 14	Pass

Test Case Analysis

Test Case 1

Input:

n = 1

Expected:

L → 0

G → 1

Total:

[
2¹ = 2
]

Result: Pass

Test Case 2

Input:

$n = 2$

Expected:

LL $\rightarrow$ 00

LG $\rightarrow$ 01

GL $\rightarrow$ 10

GG $\rightarrow$ 11

Total:

[
 $2^2 = 4$
]

Result: Pass

Test Case 3

Input:

$n = 3$

Expected number of patterns:

[
 $2^3 = 8$
]

Result: Pass

Test Case 4

Input:

$n = 4$

Expected number:

[
 $2^4 = 16$
]

Result: Pass

Test Case 5

Input:

$n = 5$

Expected number:

```
[  
2^5 = 32  
]
```

Result: Pass

Test Case 6

Input:

n = 6

Expected number:

```
[  
2^6 = 64  
]
```

Result: Pass

Test Case 7

Input:

L

Expected:

Binary = 0
Decimal = 0

Result: Pass

Test Case 8

Input:

G

Expected:

Binary = 1
Decimal = 1

Result: Pass

Test Case 9

Input:

LG

Expected:

Binary = 01
Decimal = 1

Result: Pass

Test Case 10

Input:

GLG

Expected:

Binary = 101

Decimal = 5

Result: Pass

Test Case 11

Input:

LGLG

Expected:

Binary = 0101

Decimal = 5

Result: Pass

Test Case 12

Input:

GGGG

Expected:

Binary = 1111

Decimal = 15

Result: Pass

Test Case 13

Input:

LXL

Since X is not a valid Laghu or Guru symbol:

Error: Pattern can contain only L and G.

Result: Pass

Test Case 14

Input:

Empty pattern

Expected:

Error: Pattern cannot be empty.

Result: Pass

Test Case 15

Input:

$n = 0$

Expected:

Error: Number of syllables must be a positive integer.

Result: Pass

Test Case 16

Input:

$n = -3$

Expected:

Error: Number of syllables must be a positive integer.

Result: Pass

Test Case 17

Input:

$n = 7$

Expected:

[
 $2^7 = 128$
]

Result: Pass

Test Case 18

Input:

GGGL

Expected:

Binary = 1110
Decimal = 14

Result: Pass

7. Complexity & Limitations

7.1 Complexity

For n syllable positions, there are:

[
 2^n
]

possible Laghu-Guru combinations.

Each generated pattern contains n symbols.

Therefore, the time complexity of generating all patterns is:

[
 $\boxed{O(n \times 2^n)}$
]

Space Complexity

When all generated patterns are stored:

[
 $\boxed{O(n \times 2^n)}$
]

This is because there are 2^n patterns and each pattern has n symbols.

7.2 Limitations

1. The number of patterns grows exponentially as the number of syllable positions increases.
2. Large values of n can produce a very large amount of output.
3. The binary representation is a **modern computational interpretation** of the two-state Laghu-Guru structure.
4. The project does not claim that Pingala used modern binary notation.
5. The implementation does not cover the complete rules of Sanskrit prosody.
6. The system generates combinations but does not determine whether each combination corresponds to a recognized Sanskrit metre.
7. The current implementation is command-line based.
8. Linguistic and grammatical analysis of Sanskrit is outside the scope of this implementation.

7.3 Correctness Analysis

The algorithm is correct for the defined computational model.

Each syllable position has exactly two possible states:

L $\rightarrow$ 0
G $\rightarrow$ 1

Therefore, for n positions, the number of possible combinations is:

[
 2^n
]

The algorithm generates every binary number between:

[
0
]

and:

$[2^n - 1]$

An n -bit binary sequence represents exactly one combination of Laghu and Guru.

Therefore, every possible Laghu-Guru pattern is generated exactly once.

For example, when $n = 3$, the algorithm generates:

000
001
010
011
100
101
110
111

which gives exactly:

$[2^3 = 8]$

unique patterns.

The test suite also verifies:

- Pattern generation
- Binary conversion
- Decimal conversion
- Boundary values
- Invalid patterns
- Invalid numbers of syllables

Therefore, the implementation is correct for the defined computational problem.

8. Conclusion

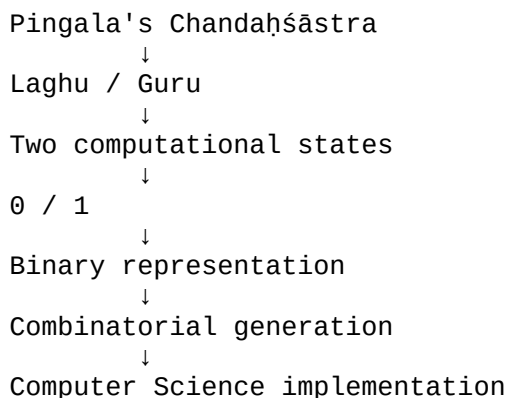
This project demonstrates how a concept associated with **Pingala's Chandaḥśāstra** can be represented using modern Computer Science concepts.

The two syllable states, **Laghu** and **Guru**, are represented computationally as 0 and 1. This enables the generation of all possible patterns using binary enumeration and combinatorial generation.

The implementation demonstrates the following CS concepts:

- Binary encoding
- Combinatorial generation
- Iteration
- Searching
- Input validation
- Mathematical representation
- Algorithm design
- Complexity analysis
- Software testing

The project successfully establishes the following mapping:



The implementation therefore provides a practical example of connecting an **Indian Knowledge System concept with a modern computational model**.

The project also maintains an important distinction between the historical knowledge system and its modern interpretation: the implementation does not claim that Pingala used modern binary notation, but demonstrates how the two-state combinatorial structure can be modeled using binary encoding.

Hence, the project fulfills the requirements of the **IKS in Computational Systems** course by providing:

1. An IKS concept mapped to a CS concept.
2. A formal specification of rules and algorithms.
3. A working Python implementation.

4. More than the required 10 test cases.
5. Correctness analysis.
6. Complexity analysis.
7. Limitations of the computational model.
8. A clear conclusion connecting IKS with Computer Science.